

Vehicle Control System (VCS) Architecture & Task Specification

This document details the concurrent multi-core task allocation, object-oriented module hierarchy, and data execution pathways optimized for the SIDLAK 2 autonomous battery-electric prototype vehicle.

I. Unified Core Allocation & FreeRTOS Task Layout

The firmware enforces a strict separation between safety-critical physical execution loops (Core 1) and asynchronous, non-deterministic communication networks (Core 0) to ensure they operate simultaneously without inducing blocking states.

1. FreeRTOS Task Control Matrix

The system manages concurrent sub-tasks through the FreeRTOS scheduler, using explicit core pinning and priority balancing.

Task Identifier	Target Core	Priority	Loop Window	Watchdog Status	Coordinated Modules
ControlTask	Core 1	5 (Highest)	10 ms	Subscribed	VCS_System (Master FSM)
ESP32_CommLoop	Core 0	4	10 ms	Subscribed	VCS_Comm (micro-ROS Gateway)
ESP32_UILoop	Core 0	2	50 ms	Subscribed	System Telemetry Loggers
DisplayLoop	Core 1	1	50 ms	Subscribed	VCS_Display
WebServerTask	Core 0	1 (Lowest)	50 ms	Unsubscribed	VCS_Simulation (Asynchronous Web Tuner)

2. Core 1: Hard Real-Time Execution (The Reflex)

This core is dedicated entirely to time-critical loops, high-frequency sensor updates, and low-latency hardware actuation. It cannot be interrupted by Wi-Fi background operations or UART serialization lines.

Critical Task Definitions

- ControlTask

- Responsibilities: Drives the master vehicle Finite State Machine (FSM). It continuously evaluates physical safety elements including the auto-mode button, brake pedal inputs, and dual-grip deadman switches to route safe execution boundaries to the physical drive layout.
- DisplayLoop
 - Responsibilities: Manages the SSD1306 OLED physical diagnostic layout over the I2C bus. To minimize processing time, it reads pre-calculated, cached data arrays updated by the core routines instead of calculating parameters inside its own loop.

Encapsulated Core 1 Hardware Modules

- VCS_Steering (Closed-Loop Guidance):
 - Reads the absolute feedback voltage from the 3590S steering potentiometer via calibrated eFuse ADC channels.
 - Applies a localized STEER_EMA_ALPHA filter to clean noise, computes the targeted PID response, and streams glitch-free step pulses directly to the DM542 driver.
- VCS_Actuators (Power & Delivery):
 - Monitors the status of the physical auto-mode button.
 - Scales target speeds to an 8-bit DAC output voltage on THROTTLE_DAC_PIN for motor controller delivery, commands the TB6612FNG H-bridge directional states for timed brake strokes, and triggers the physical safety mode-switching relays.
- VCS_Hall (Telemetry Source):
 - Catches rising-edge hardware interrupts generated by the wheel magnet speed sensor line (HALL_C_PIN) to compute true vehicle velocity and wheel RPM.
- VCS_Reverse (Directional Awareness):
 - Evaluates latching switch states on PIN_REVERSE_IN and validates direction shifts only if measured wheel speeds are below the safe entry parameter (STOPPED_ENTER_RPM < 3).

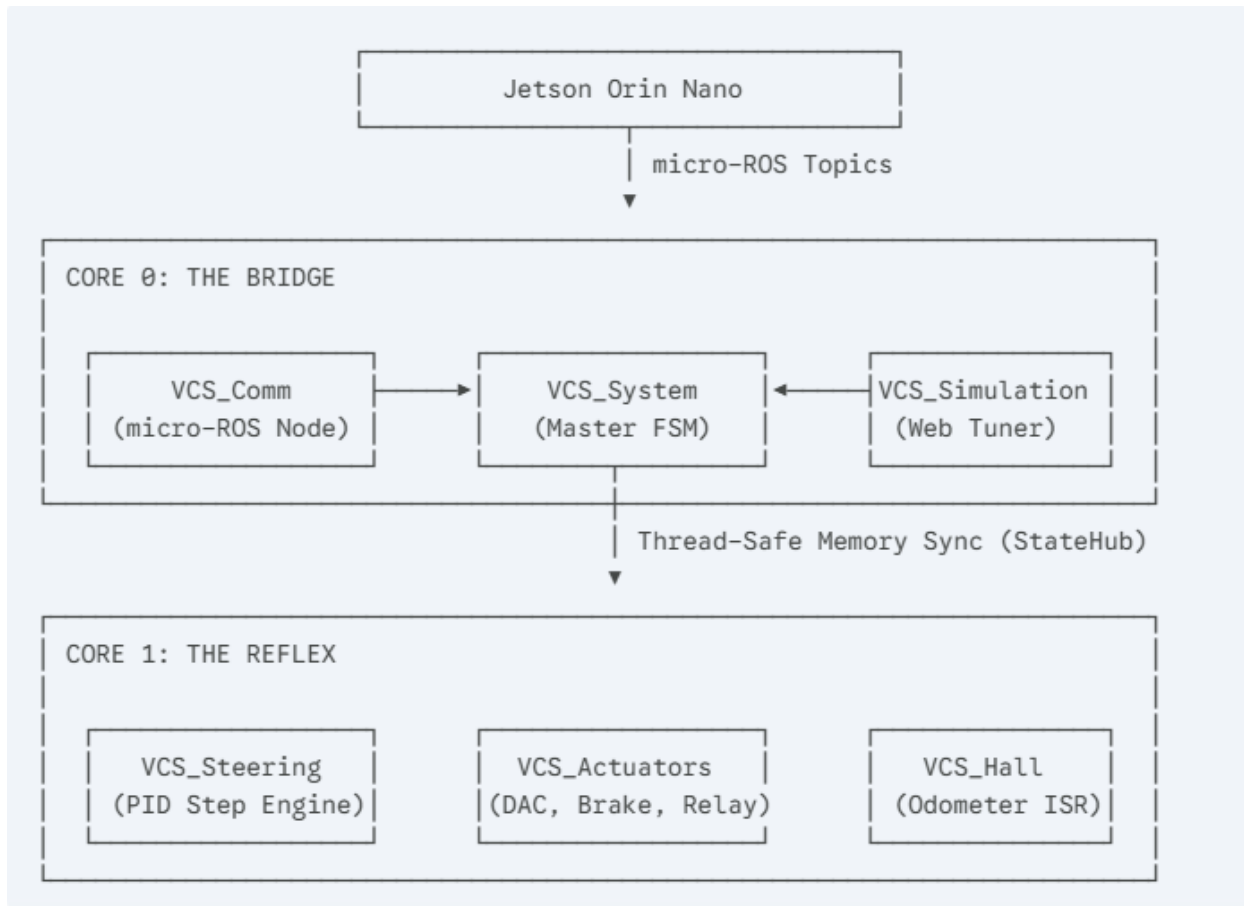
3. Core 0: Communication, Simulation & Vis (The Bridge)

This core isolates background network interactions, serial parsing tasks, and data streaming layers from the core hardware drives.

Critical Task Definitions

- **ESP32_CommLoop**
 - **Responsibilities:** Manages the micro-ROS native execution client. It intercepts structured command arrays from the Jetson Orin Nano, extracts normalized steering/throttle percentages, and packages physical odometry metrics into standard ROS 2 topic streams back to the host.
- **ESP32_UILoop**
 - **Responsibilities:** Handles high-level diagnostics and manages data structures for vehicle telemetry recording.
- **WebServerTask**
 - **Responsibilities:** Spins up the localized Wi-Fi Access Point (SIDLAK_VCS_LIVE) and maintains high-speed WebSocket endpoints to stream target versus actual vehicle metrics at 20Hz directly to your web tuning dashboard.

II. Overall VCS Module Summary



VCS_System (The Core FSM Hub)

- **Role:** The system conductor. It holds the global state machine structure and runs transition checks. It evaluates physical button parameters, safety overrides, and incoming micro-ROS control frames to assign vehicle movement authorization.

VCS_Comm (The micro-ROS Gateway)

- **Role:** Oversees the high-speed serial transport framework. It captures command message arrays from the autonomous navigation stack, performs CRC16 check validations, and passes frames into the shared state registers.

VCS_Simulation (The Asynchronous Web Tuner)

- **Role:** Manages the web dashboard interface. It processes parameters received over WebSockets to let you track real-time target versus actual operational curves during track testing.

VCS_Display (Local Diagnostics)

- **Role:** Controls the cockpit OLED hardware. It outputs warning registers, system fault statuses, connection indicators, and battery tracking details directly to the operator.

VCS_Config (The Flash Memory Vault)

- **Role:** Encapsulates persistence loops via Preferences.h. It saves updated loop metrics, scaling coefficients, and filter constants permanently to non-volatile storage (NVS). To protect the processor from write-latency stalls, it locks flash writing operations whenever the FSM enters an autonomous driving state.

III. Data Flow Execution Model

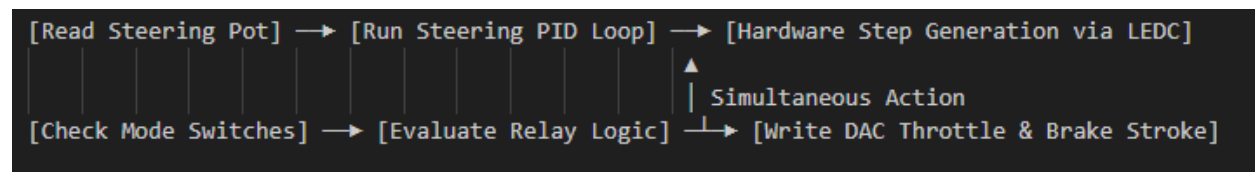
Simultaneous execution across the dual-core microcontroller is achieved through split, non-blocking asynchronous calculation pathways.

1. Perception/Comms Cycle (Core 0 Framework)

[Jetson Command Frame] → [VCS_Comm Parses Array] → [Update VCS_System Cache] → [WebSocket Telemetry Push]

1. The VCS_Comm gateway intercepts fresh navigation packets from the Jetson Orin Nano.
2. Incoming values are checked and validated.
3. The system stores the current position metrics into a single-tick global tracker (cachedSteering). This design prevents multiple subsystems from applying internal slew-rate limit filters iteratively within the same cycle, ensuring data consistency.
4. The system updates the targets inside the master state hub (VCS_System).
5. VCS_Simulation updates its wireless data registers and pushes matched JSON frames out to connected WebSocket diagnostic browsers.

2. Actuation/Sensor Cycle (Core 1 Framework)



1. The 1000Hz hardware timer loop alerts the processor to begin operations.
2. VCS_Steering captures raw analog feedback signals from the potentiometer. It processes these readings against the target values stored in the state hub and calculates the required frequency corrections for the stepper motor.

3. Simultaneously, VCS_Actuators verifies the active safety mode, checks physical button lines, and manages the safety interlock relays.
4. The core outputs matched control signals: sending scaled driving voltages to the 8-bit throttle DAC, adjusting the TB6612FNG H-bridge outputs to shift the brake linear actuator, and recording new speed values generated by the wheel magnet sensor interrupts.